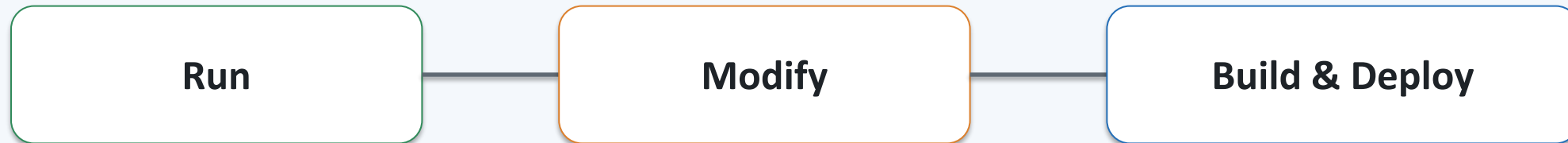


Docker Part 3: Hands-on Practical

Ubuntu • Custom Image • Apache • Bind Mount • Nginx • Dockerfile



Practical Roadmap

Practical 1 — Ubuntu container and container lifecycle

Practical 2 — Install Python and create a custom image with docker commit

Practical 3 — Deploy Apache with port mapping and customize the web page

Practical 4 — Share web files using a bind mount

Practical 5 — Deploy Nginx and run two containers with shared content

Practical 6 — Build a reproducible custom image using Dockerfile

Before the Practical

Check Docker and make sure the current user can run Docker commands.

```
docker --version  
docker ps  
docker images
```

If Docker is not running: `sudo systemctl enable --now docker`

If permission was just added with `usermod`, log out/in or reboot before trying again.

For this class, Ubuntu VM networking can remain NAT.

Practical 1 — Pull Ubuntu and Create a Container

```
docker pull ubuntu  
docker images  
docker run --name u1 -it ubuntu bash
```

The image is downloaded once and stored locally.

docker run creates a new container named u1.

-it gives an interactive terminal; bash runs inside the container.

ubuntu image

u1 container

Practical 1 — Work Inside the Container

```
# Inside u1  
pwd  
ls  
mkdir dai  
cd dai  
touch ml  
ls  
exit
```

Create a directory and file inside the container.

exit stops the interactive bash process, so the container will normally be removed.

Back on the host, verify its status.

```
docker ps  
docker ps -a
```

Practical 1 — Restart and Re-enter u1

```
docker start u1
docker exec -it u1 bash
cd /dai
ls
exit
```

docker start restarts the existing container; it does not create a new one.

docker exec opens a new bash process in the running container.

The dai directory and ml file should still be present because the container is persistent.

```
docker stop u1
docker rm u1
```

Cleanup after verification.

Practical 2 — Customize Ubuntu with Python

```
docker run --name ub1 -it ubuntu bash
```

```
# Inside ub1  
apt update  
apt install python3 -y  
python3 --version  
python3
```

```
>>> print("Hello, welcome to Docker")  
# Ctrl+D to leave Python  
exit
```

We manually modify the writable container by installing Python. This prepares the container for the docker commit demonstration. docker commit is useful for learning, but Dockerfile is preferred for

Practical 2 — Create a Custom Image with docker commit

```
docker ps -a  
docker commit ub1 my-python  
docker images
```

```
docker run --name py1 -it my-python bash  
python3 --version  
python3
```

**ub1
container**

**my-python
image**

A new container created from my-python should already con

Practical 3 — Deploy Apache Web Server

```
docker pull httpd
docker run -d --name web1 -p 8000:80 httpd
docker ps
```

-d runs Apache in the background.

-p 8000:80 publishes host port 8000 to container port 80.

Open Ubuntu Firefox and browse to <http://localhost:8000>.

Expected result: Apache default page.



Practical 3 — Customize Apache with docker cp

```
cd ~  
nano index.html
```

```
<h1>Welcome to the world of containers!</h1>
```

```
docker cp index.html web1:/usr/local/apache2/htdocs/index.html
```

Refresh <http://localhost:8000> — your custom page should replace the default page.

`docker cp` copies a snapshot of the file; later edits on the host are not automatically synchronized.

Practical 4 — Apache Bind Mount

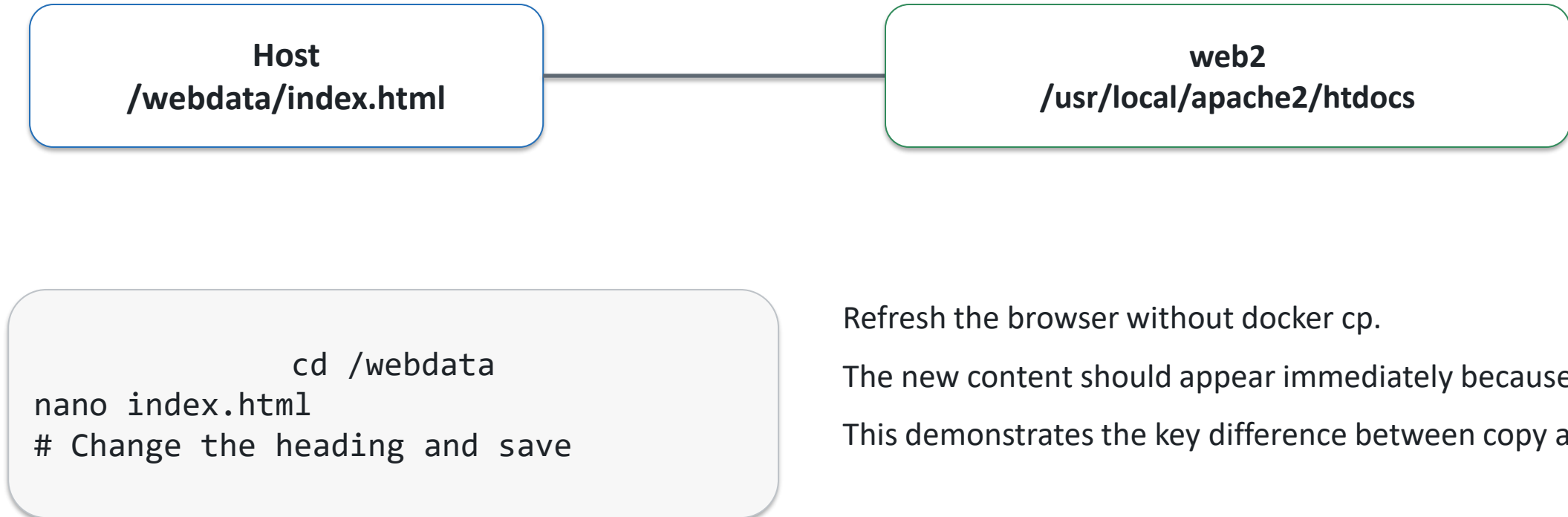
```
sudo mkdir -p /webdata  
sudo chown $USER:$USER /webdata  
cd /webdata  
nano index.html
```

```
<h1>This site is under construction!</h1>
```

```
docker stop web1  
  
docker rm web1  
docker run -d --name web2 -p 9000:80 \  
-v /webdata:/usr/local/apache2/htdocs \  
httpd
```

Browse: <http://localhost:9000>

Practical 4 — Verify Live File Sharing



Practical 5 — Deploy Nginx

```
docker pull nginx
docker run -d --name c1 -p 4500:80 nginx
docker ps
```

Browse to <http://localhost:4500>.

Expected result: the default Nginx welcome page.

Nginx's default web root in the official image is `/usr/share/nginx`

```
cd ~
nano index.html
docker cp index.html
c1:/usr/share/nginx/html/index.html
```

Refresh the browser to verify the custom page.

Practical 5 — Two Nginx Containers, One Shared Folder

```
docker stop c1
docker rm c1
cd ~
mkdir -p exam-data
cd exam-data
nano index.html
```

```
docker run -d --name ng1 -p 7500:80 \
-v "$(pwd)":/usr/share/nginx/html nginx

docker run -d --name ng2 -p 7600:80 \
-v "$(pwd)":/usr/share/nginx/html nginx
```

exam-data

ng1 :7500

ng2 :7600

Browse both ports. Both containers should display

Practical 5 — Modify Once, Update Both

```
cd ~/exam-data  
nano index.html  
# Change the page and save
```

```
docker stop ng1 ng2  
docker rm ng1 ng2
```

Refresh <http://localhost:7500> and <http://localhost:7600>.

Both pages update because ng1 and ng2 mount the same host directory.

Different host ports are required because two running containers cannot

Practical 6 — Why Dockerfile?

docker commit captures a manually modified container as an image.

A Dockerfile records the image-building steps as text.

This makes the build easier to repeat, review, share, and automate.

```
FROM ubuntu
RUN apt-get update && apt-get install -y
python3 \
    && rm -rf /var/lib/apt/lists/*
CMD ["python3", "--version"]
```

Dockerfile

docker build

Image

Practical 6 — Build and Run the Dockerfile

```
mkdir -p ~/docker-python  
cd ~/docker-python  
nano Dockerfile
```

```
FROM ubuntu  
RUN apt-get update && apt-get install -y python3 \  
    && rm -rf /var/lib/apt/lists/*  
CMD ["python3", "--version"]
```

```
docker build -t my-python:v1 .  
docker images  
docker run --rm my-python:v1
```

Expected final output includes the installed Python 3 version.
--rm automatically removes this temporary container after it exits.

Practical Review: What Students Have Done

Pulled Docker images and created containers.

Worked inside an interactive Ubuntu container.

Started, stopped, re-entered, and removed containers.

Created a custom image using docker commit.

Published Apache and Nginx web servers with host-to-container port mapping.

Used docker cp and bind mounts for web content.

Shared one host directory with two containers.

Built a reproducible image from a Dockerfile.

Next Step: Toward Kubernetes / Minikube



After Docker fundamentals, students are ready to learn why container orchestration is needed and how Minikube provides a local Kubernetes environment.